

**PRACTICE EXERCISES OF THE MICROPROCESSORS
& MICROCONTROLLERS**

Instructor: The Tung Than

Student's name: Gia Hung Nguyen

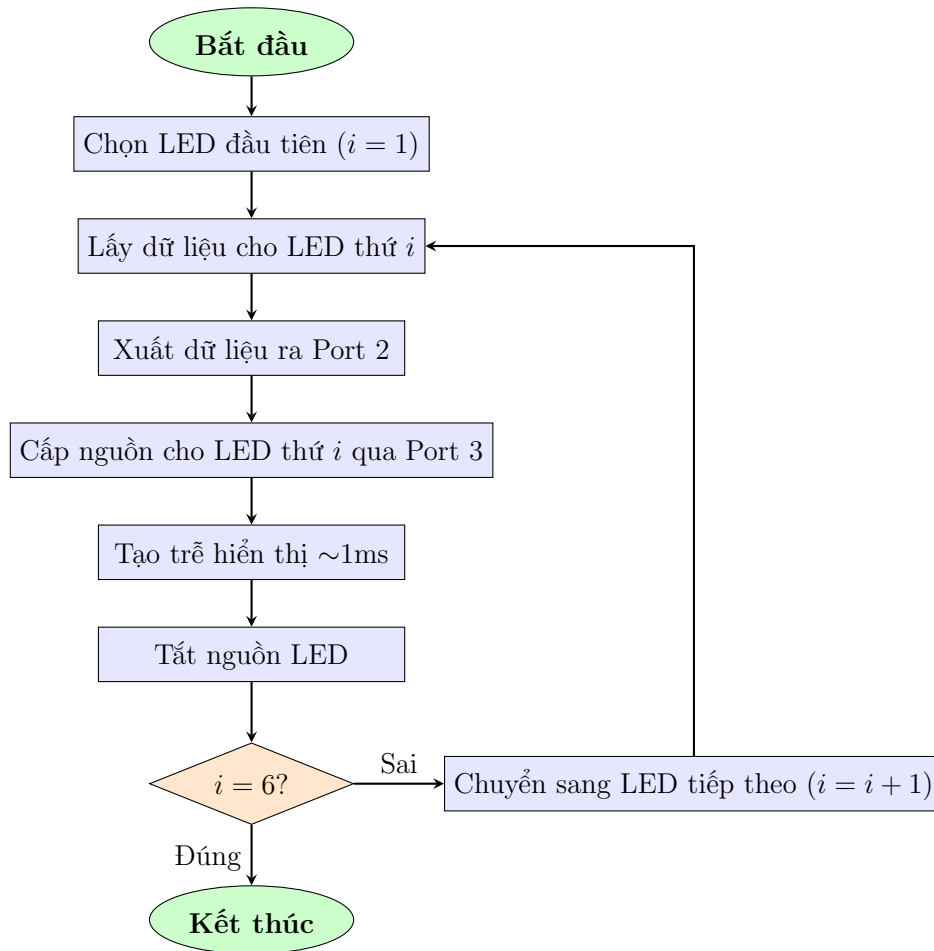
Student code: 24520603

PRACTICE REPORT NO 2
**COMMUNICATION WITH 7-SEGMENT LED
AND TIMER**

- I. Lưu đồ thuật toán quét LED sử dụng LED 7 đoạn**
- II. Thiết kế**
- III. Xây dựng chương trình Assembly**
- IV. So sánh hai phương pháp delay bằng Timer và Vòng lặp**

I. Lưu đồ thuật toán quét LED sử dụng LED 7 đoạn

1. Lưu đồ



2. Giải thích lưu đồ

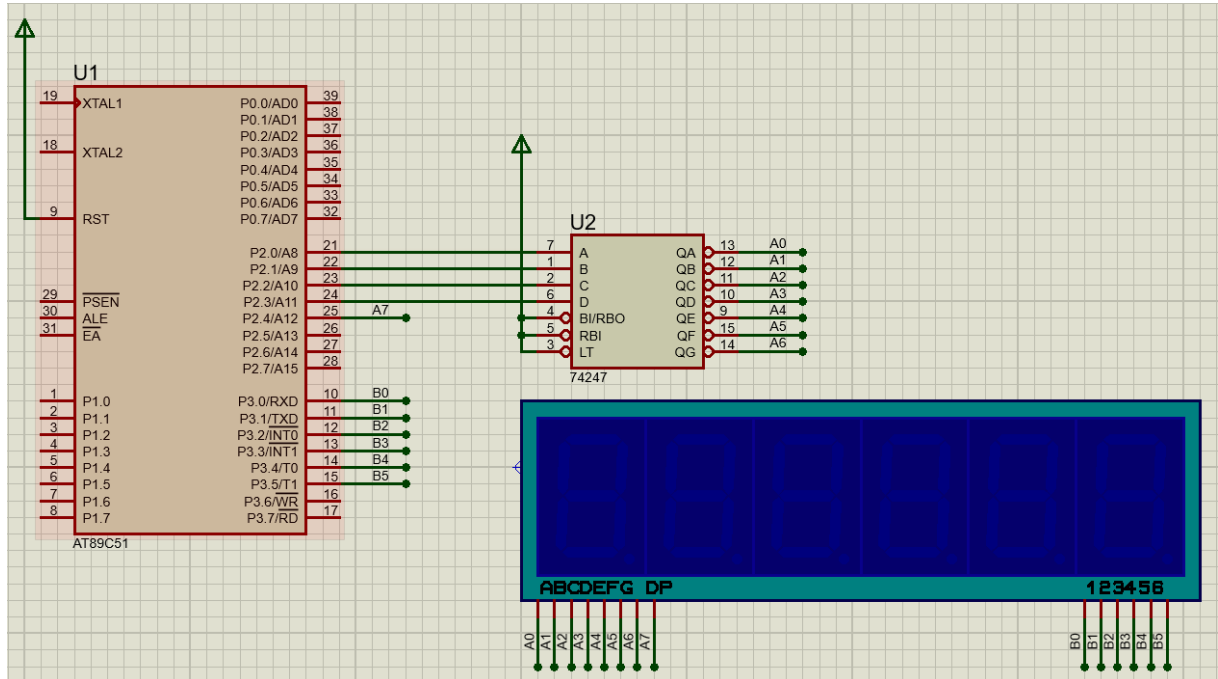
Lưu đồ mô tả thuật toán quét lần lượt 6 LED 7 đoạn trong một chu kỳ:

- **Khởi tạo:** Bắt đầu quét bằng việc chọn vị trí LED đầu tiên ($i = 1$).
- **Hiển thị:** Lấy dữ liệu (mã 7 đoạn) của LED thứ i xuất ra Port 2, sau đó cấp nguồn qua Port 3 để làm sáng LED tương ứng.
- **Tạo trễ:** Duy trì trạng thái sáng khoảng 1ms để mắt người kịp nhìn thấy. Sau đó, tắt nguồn LED ngay lập tức để tránh hiện tượng bóng ma khi chuyển sang LED tiếp theo.
- **Kiểm tra vòng lặp ($i = 6?$):**
 - Nếu **Sai**: Chưa quét hết 6 LED, tiến hành tăng biến đếm ($i = i + 1$) để chuyển sang LED kế tiếp và lặp lại quá trình.
 - Nếu **Đúng**: Đã quét đủ 6 vị trí LED, kết thúc.

II. Thiết kế

Mô phỏng được thực hiện trên phần mềm Proteus với các thành phần chính:

- **Khối điều khiển trung tâm:** Vi điều khiển họ 8051 (IC AT89C51).
- **Khối hiển thị:** Gồm 6 LED 7 đoạn dùng để hiển thị. Dữ liệu từng chữ số được xuất qua **Port 2** nối với decoder, trong khi tín hiệu chọn LED được điều khiển thông qua **Port 3**.



Hình 1: Sơ đồ mạch đồng hồ số 24h với 6 LED 7 đoạn trên Proteus

III. Xây dựng chương trình Assembly

1. Chương trình Assembly

Chương trình điều khiển đồng hồ số sử dụng kỹ thuật **quét LED** kết hợp ngắt định thời bằng **Timer 0**:

- **Cấu trúc thời gian:** Sử dụng các thanh ghi R1, R2 (Giờ), R3, R4 (Phút), R5, R6 (Giây) để lưu trữ các chữ số BCD.
- **Quét LED & Định thời 1 giây:** Mỗi vi mạch LED được bật sáng trong 1ms (định thời bằng Timer 0). Một vòng quét đủ 6 LED mất 6 ms. Để tạo ra thời gian thực 1 giây (1000 ms), hệ thống dùng biến R0 để lặp lại vòng quét 167 lần ($167 \times 6 \text{ ms} = 1002 \text{ ms} \approx 1 \text{ giây}$).

- **Dấu chấm phân cách (DP):** Mạch LED tích cực mức thấp. Tại các vị trí phân cách, dữ liệu xuất ra không bị can thiệp để làm **sáng** dấu chấm. Tại các LED còn lại, dùng lệnh `ORL A, #00010000b` để ép bit điều khiển DP lên mức 1, qua đó **tắt** dấu chấm.

ASSEMBLY	GIẢI THÍCH
<pre> ORG 0000H LJMP MAIN ORG 0100H MAIN: RESET_CLOCK: MOV R1, #0 MOV R2, #0 MOV R3, #0 MOV R4, #0 MOV R5, #0 MOV R6, #0 </pre>	RESET_CLOCK: Reset lại toàn bộ thời gian khi đếm đủ 24h. Nạp giá trị 0 vào các thanh ghi R1 đến R6. Đồng hồ bắt đầu chạy từ mốc 00.00.00.
<pre> HIENTHI: MOV R0, #167 LOOP_QUET: MOV A, R6 ORL A, #00010000b MOV P2, A MOV P3, #00100000b ACALL DELAY_1MS_TIMER/LOOP MOV P3, #0 MOV A, R5 ORL A, #00010000b MOV P2, A MOV P3, #00010000b ACALL DELAY_1MS_TIMER/LOOP MOV P3, #0 </pre>	Biến đếm 1 giây: Nạp R0 = 167. Mỗi vòng quét trễ 6ms → 167 vòng ≈ 1 giây. Quét LED phần Giây (R6, R5): ORL A, #00010000b: Ép bit 4 lên mức 1 để tắt dấu chấm DP. MOV P2, A: Xuất dữ liệu ra decoder nối với Port 2. MOV P3, #...: Cấp nguồn cho LED tương ứng sáng. ACALL DELAY_1MS_TIMER/LOOP: Gọi hàm delay 1ms. MOV P3, #0: Tắt hoàn toàn LED trước khi chuyển sang LED khác để chống hiện tượng bóng ma.

ASSEMBLY	GIẢI THÍCH
<pre> MOV P2, R4 MOV P3, #00001000b ACALL DELAY_1MS_TIMER/LOOP MOV P3, #0 MOV A, R3 ORL A, #00010000b MOV P2, A MOV P3, #00000100b ACALL DELAY_1MS_TIMER/LOOP MOV P3, #0 </pre>	Quét LED phần Phút (R4, R3): Tại LED Đơn vị phút (R4), dữ liệu được xuất thẳng ra Port 2 mà không qua lệnh ORL. Do đó bit điều khiển DP giữ nguyên mức 0 → Dấu chấm phân cách sáng lên. Tại LED Chục phút (R3), dấu chấm không cần thiết nên lại tiếp tục dùng lệnh ORL để tắt đi, sau đó quét và tạo trễ tương tự.
<pre> MOV P2, R2 MOV P3, #00000010b ACALL DELAY_1MS_TIMER/LOOP MOV P3, #0 MOV A, R1 ORL A, #00010000b MOV P2, A MOV P3, #00000001b ACALL DELAY_1MS_TIMER/LOOP MOV P3, #0 DJNZ R0, LOOP_QUET </pre>	Quét LED phần Giờ (R2, R1): Tương tự như R4, LED Đơn vị giờ (R2) được xuất trực tiếp để làm sáng dấu chấm phân cách. LED Chục giờ (R1) tắt dấu chấm và quét bình thường. DJNZ R0, LOOP_QUET: Sau khi quét xong 1 lượt 6 LED, giảm biến đếm R0 đi 1. Nếu R0 chưa về 0 thì lặp lại quy trình quét. Khi R0 = 0 (đã trôi qua 1 giây), thoát vòng quét để chuyển sang cập nhật thời gian.

ASSEMBLY	GIẢI THÍCH
TIME: INC R6 CJNE R6, #10, HIENTHI MOV R6, #0 INC R5 CJNE R5, #6, HIENTHI MOV R5, #0 INC R4 CJNE R4, #10, HIENTHI MOV R4, #0 INC R3 CJNE R3, #6, HIENTHI MOV R3, #0 INC R2	INC Rx: Tăng giá trị thanh ghi. CJNE Rx, #Giá_trị, HIENTHI: Kiểm tra giới hạn. Nếu chưa đạt giới hạn (10 đối với hàng đơn vị, 6 đối với hàng chục) thì nhảy thẳng về HIENTHI để bắt đầu chu kỳ 1 giây mới. Nếu đạt giới hạn, reset thanh ghi về 0 và chạy lệnh INC ở thanh ghi tiếp theo để chuyển sang hàng lớn hơn.
CJNE R1, #2, CHECK_24H CJNE R2, #4, CHECK_24H LJMP RESET_CLOCK CHECK_24H: CJNE R2, #10, HIENTHI MOV R2, #0 INC R1 LJMP HIENTHI	Kiểm tra đồng thời R1 và R2. Nếu R1 = 2 và R2 = 4 (đồng hồ điểm 24h00), nhảy về RESET_CLOCK để xóa về 0. CHECK_24H: Nếu chưa tới 24h00, kiểm tra xem đơn vị giờ (R2) đã đạt 10 chưa. Nếu chưa thì nhảy về HIENTHI. Nếu đủ 10 thì reset R2 = 0, tăng chục giờ (R1) và nhảy về HIENTHI (tránh lỗi trôi code xuống hàm Delay).

ASSEMBLY	GIẢI THÍCH
DELAY_1MS_TIMER: MOV TMOD, #01H MOV TH0, #0FCH MOV TL0, #018H CLR TF0 SETB TR0 JNB TF0, \$ CLR TR0 RET	Hàm định thời 1ms bằng Timer 0: MOV TMOD, #01H: Khởi tạo Timer 0 hoạt động ở Chế độ 1. Nạp giá trị đếm: Nạp TH0 = FCH và TL0 = 18H → Hệ thập phân là 64536. Số chu kỳ máy cần đếm tới khi tràn cờ TF0 là: $65536 - 64536 = 1000$. Với tần số thạch anh cơ bản, 1 chu kỳ máy = $1 \mu s$, ta có $1000 \times 1 \mu s = 1000 \mu s = 1 ms$. JNB TF0, \$: Dừng tại chỗ chờ cờ tràn dựng lên. CLR TR0: Dừng Timer khi đã đủ thời gian và RET thoát hàm.
DELAY_1MS_LOOP: PUSH 00H PUSH 01H MOV R0, #2 LOOP1: MOV R1, #100 CONT: DJNZ R1, LOOP2 DJNZ R0, LOOP1 POP 01H POP 00H RET LOOP2: NOP JMP CONT END	Hàm định thời 1ms bằng vòng lặp: PUSH/POP: Cất tạm giá trị R0 và R1 vào Stack để tránh bị ghi đè dữ liệu, sau đó lấy ra trả lại nguyên vẹn khi kết thúc delay. LOOP2: Lệnh NOP (1 chu kỳ) và JMP (2 chu kỳ) tốn tổng cộng 3 chu kỳ. CONT: Lệnh DJNZ R1 tốn 2 chu kỳ → Mỗi lần lặp con tốn $3 + 2 = 5$ chu kỳ. Số chu kỳ lặp: $(R1 = 100, R0 = 2) \rightarrow 100 \times 5 \times 2 = 1000$ chu kỳ. Với 1 chu kỳ máy = $1 \mu s \rightarrow$ Tổng thời gian delay = $1000 \mu s = 1ms$.

2. Mô phỏng LED trên Proteus

Link video mô phỏng và File thiết kế: https://drive.google.com/drive/folders/1GL0jXI1IF_Q2LqHbYh8TL0wBvZo_67kV?usp=sharing

IV. So sánh hai phương pháp delay bằng Timer và Vòng lặp

Phương pháp	Ưu điểm	Nhược điểm
Timer	- Độ chính xác cực kỳ cao, dễ dàng thiết lập thời gian chuẩn thông qua công thức toán học. - Timer hoạt động độc lập với CPU. Nếu sử dụng thêm Ngắt, CPU có thể làm việc khác trong lúc chờ.	- Tiêu tốn tài nguyên phần cứng của vi điều khiển (mất 1 bộ Timer). - Cần thao tác với nhiều thanh ghi chuyên dụng (TMOD, THx, TLx, TRx, TFX) nên code phức tạp hơn.
Vòng lặp	- Code đơn giản, dễ hiểu, chỉ dùng các thanh ghi đa dụng (R0-R7). - Không sử dụng Timer phần cứng, giải phóng Timer cho các tác vụ quan trọng khác (như tạo xung PWM, giao tiếp UART...).	- CPU bị chiếm dụng hoàn toàn chỉ để thực hiện việc đếm lùi, gây lãng phí hiệu năng. - Ít chính xác hơn. Mỗi khi thay đổi thạch anh hoặc thêm bớt lệnh trong code, phải tính toán lại số lần lặp thủ công.